



TEMA IV

DESARROLLO DE APLICACIONES WEB EN BACKEND

*"Siempre parece imposible,
hasta que se hace"*
Nelson Mandela

8. PANEL DE ADMINISTRACIÓN

Un panel de administración nos sirve para gestionar aplicaciones web en las que hay gestión de usuarios, mucho contenido que cambia continuamente, un front potente...

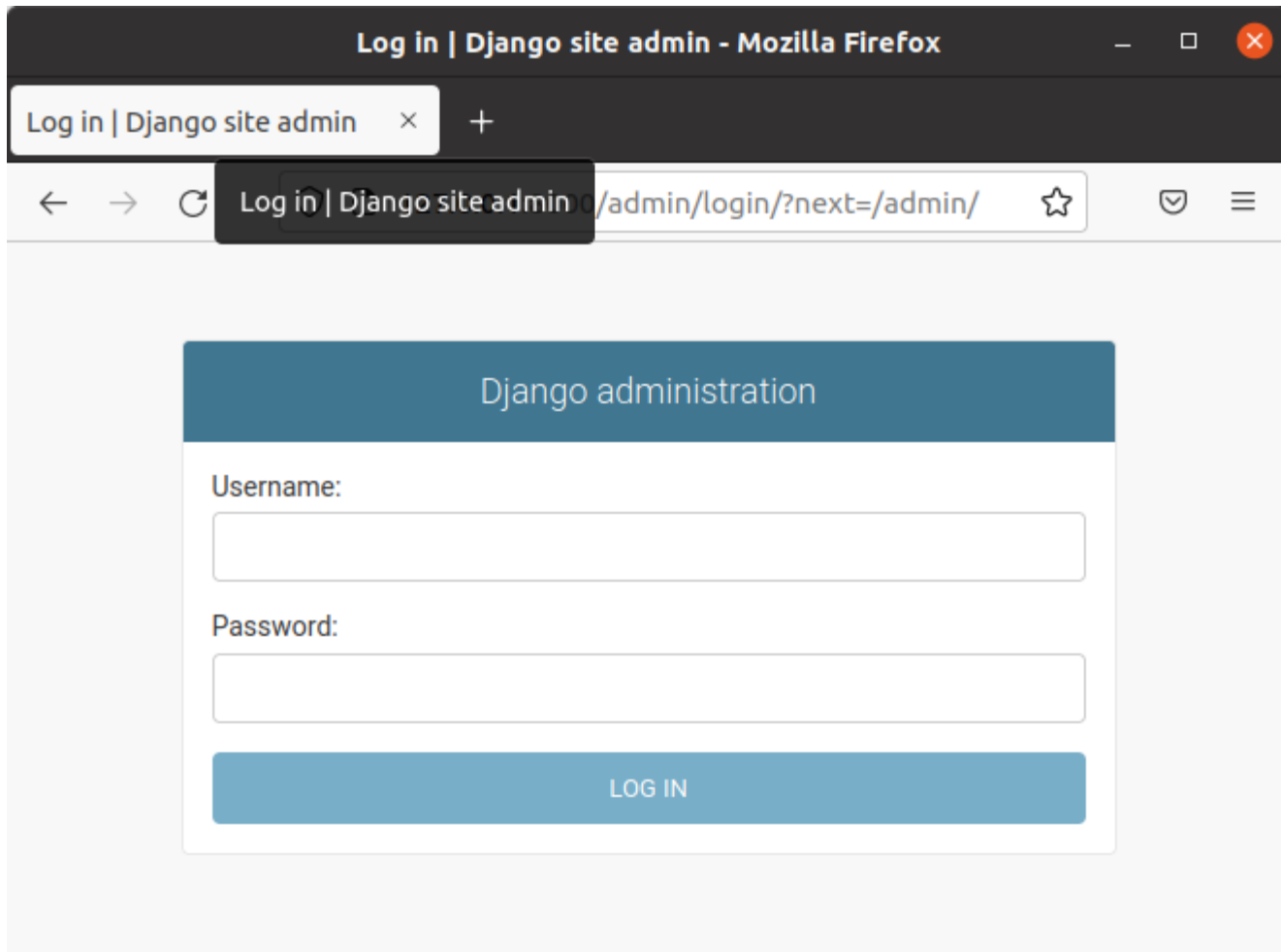
Al crear un proyecto ya tenemos habilitado por defecto el panel de administración, de echo tanto en el archivo settings como en url ya tenemos referencias al mismo

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
]
```

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
]
```

Así mismo al crear una app dentro del proyecto nos agrega el archivo admin.py

Si arranco el proyecto en mi navegador y le añado en la url /admin arrancamos sin problema el panel de administración



Para acceder necesitamos crear un superusuario con perfil de administrador para acceder al panel.

Ya veremos que podremos hacer muchas operaciones sin necesidad de hacerlo nosotros como en los temas anteriores.

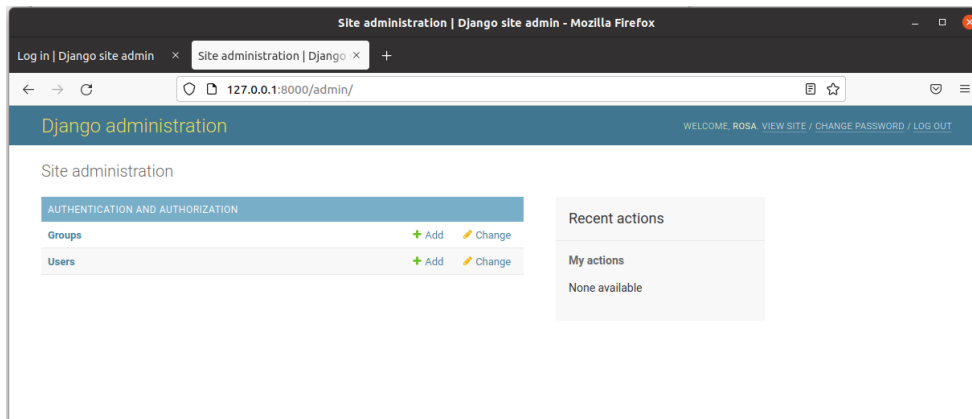
Para crear el **superusuario**, en la carpeta del proyecto

```
python3 manage.py createsuperuser
```

Una vez ejecutado, nos pedirá nombre y password del mismo.

```
rosa@rosa-Essential14L:~/Escritorio/DJANGO/panel$ python3 manage.py createsuperuser
Username (leave blank to use 'rosa'): rosa
Email address: rrodriguez@salesianasnpilar.org
Password:
Password (again):
Superuser created successfully.
rosa@rosa-Essential14L:~/Escritorio/DJANGO/panel$
```

Ahora podemos introducir nuestras credenciales y entrar en el panel



De momento sólo tengo grupos y usuarios.

Todas las tablas que nos genera la bbdd cuando migramos, son tablas que usará el panel de administración. Si nos vamos a la de usuarios debe aparecer el que acabamos de crear.

Estructura					
Hoja de datos					
Editar pragmas					
Ejecutar SQL					
Tabla: auth_user					
Nuevo registro, Borrar registro					
id	password	last_login	is_superuser	username	last_
Filtro	Filtro	Filtro	Filtro	Filtro	Filtro
1	pbkdf2_sha...	2022-01-25 ...	1	rosa	

Veamos como manipular las tablas de nuestra bbdd desde el panel.

Para ello utilizaremos el EjemploBBDD que ya teníamos operativo y vamos a añadir nuevos alumnos desde el panel:

1. En el archivo admin.py tenemos que importar los modelos que creamos y que queremos utilizar desde el panel.

```
from django.contrib import admin
from gestion_alumnos.models import Alumno
# Register your models here.

admin.site.register(Alumno)
```

Creamos un superusuario y cargamos el panel de administración, veremos que nos ha añadido la tabla correspondiente. Django añade una s al nombre

Django administration

Site administration

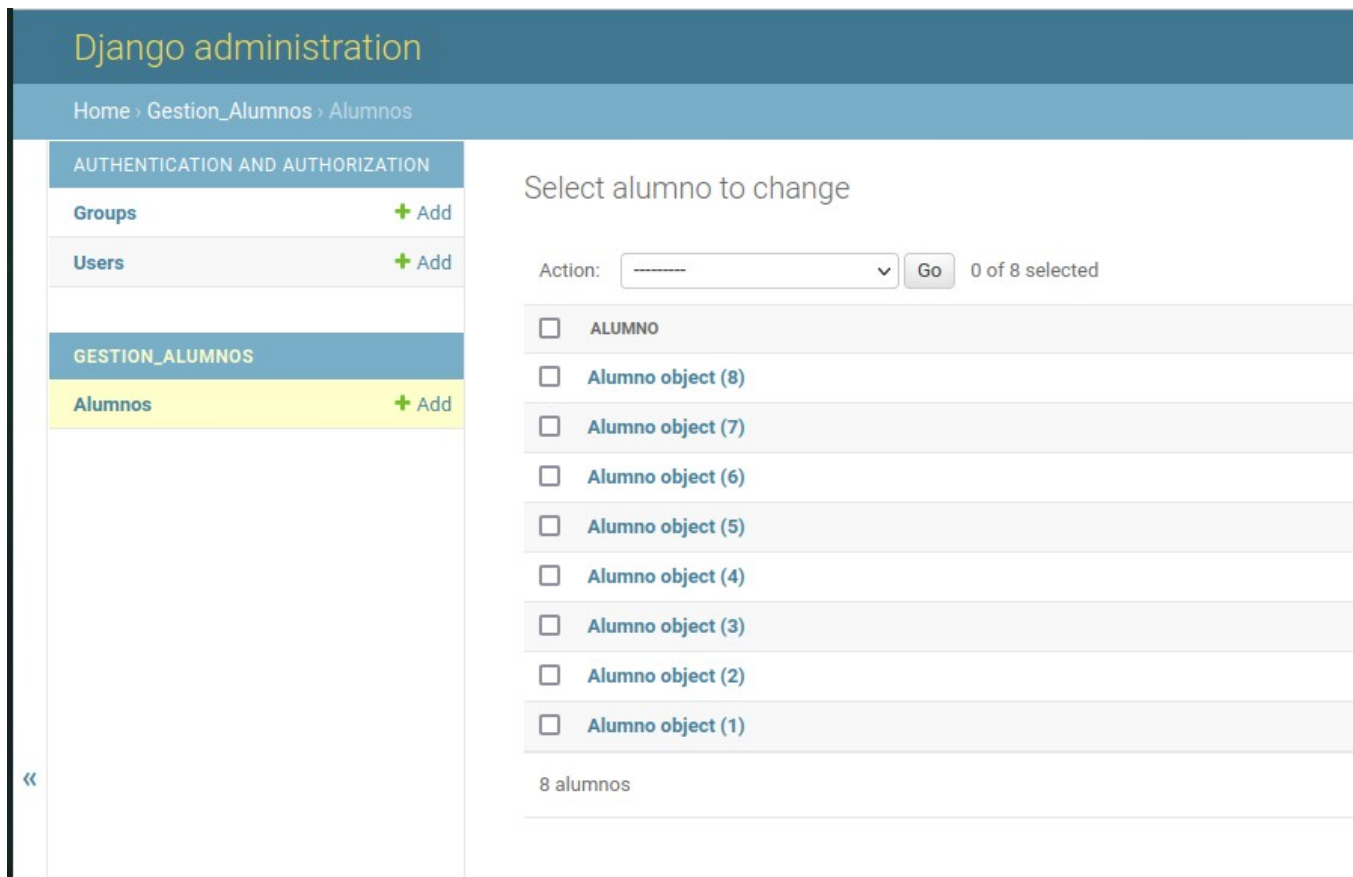
AUTHENTICATION AND AUTHORIZATION	
Groups	+ Add Change
Users	+ Add Change
GESTION_ALUMNOS	
Alumnos	+ Add Change

Recent actions

My actions

None available

Si clikeamos en la tabla Alumnos veremos que accedemos a la información de la tabla de nuestra bbdd que construimos a partir de ese modelo



Obviamente podemos añadir alumnos, modificarlos,....

Podemos añadir tantos modelos como tablas queramos gestionar desde nuestro panel. Por defecto, al crear el modelo si no decimos nada todos los campos son obligatorios, veamos como cambiar ésto: En el modelo añadimos lo siguiente

email=EmailField(max_length=50,blank=True,null=True)

Al hacer cambios en el modelo que afectarán a la tabla tenemos que migrar de nuevo (makemigrations,migrate)

Django administration

Home > Gestion_Alumnos > Alumnos > Add alumno

AUTHENTICATION AND AUTHORIZATION

- Groups + Add
- Users + Add

GESTION_ALUMNOS

- Alumnos + Add

«

Add alumno

Please correct the errors below.

Nombre: This field is required.

Dni: This field is required.

Email:

Nota: This field is required.

En la figura anterior vemos que nos muestra el listado de los objetos que tiene almacenados, si tuviéramos en nuestra clase Alumno un método `str` la información que aparecería sería el string que nos retorna ese método

```
# Create your models here.
class Alumno(models.Model):
    nombre=CharField(max_length=80)
    dni=CharField(unique=True,max_length=9)
    email=EmailField(max_length=50,blank=True,null=True)
    nota=DecimalField(max_digits=4,decimal_places=2)
    numero=models.AutoField(primary_key=True)

    def __str__(self):
        return self.nombre
```

Volvemos al panel, refrescamos y comprobamos

Select alumno to change

Action: 0 of 9 selected

☐	ALUMNO
☐	Beatriz
☐	Hola
☐	Pepe
☐	Juan Pérez
☐	Laura
☐	Pepe
☐	aa
☐	Rosa
☐	Rosa

9 alumnos

Personalizar el administrador

Es frecuente que los nombres de las app y los atributos de los modelos así como las clases estén en inglés en nuestro código. Si es el caso, podemos personalizar el panel para que en cualquier caso nos aparezca en castellano.

Por ejemplo, haremos un ejercicio en el que nuestra aplicación se llamará Portfolio en inglés y quizá queremos que en el administrador aparezca Portafolio en español. Para lograrlo hay que cambiar dos cosas,

- Añadir un campo `verbose_name` en el fichero `portfolio/apps.py`:

```
portfolio/apps.py

from django.apps import AppConfig

class PortfolioConfig(AppConfig):
    name = 'portfolio'
    verbose_name = 'Portafolio'
```

De esta forma podemos establecer una configuración extendida.

- Establecer esta configuración en `settings.py`, lo cual se hace llamando a esta clase `PortfolioConfig` en lugar de `portfolio` a secas:

```
webpersonal/settings.py

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'core',
```



```
        'portfolio.apps.PortfolioConfig',  
    ]
```

Al crear la clase `Proyecto` hemos decidido ponerle `Project` para seguir una lógica en todo el proyecto, pero podemos cambiar el nombre que se muestra en el panel de forma muy sencilla creando una subclase con `Meta`

```
portfolio/models.py  
  
from django.db import models  
  
class Project(models.Model):  
    title = models.CharField(max_length=200)  
    .....  
    class Meta:  
        verbose_name = "proyecto"  
        ordering = ["-created"]
```

También es aconsejable poner un campo de ordenación por defecto para nuestros registros, que en nuestro caso podría ser la fecha de creación.

`Ordering` es una lista porque permite ordenar con prioridades entre distintos campos. Además si añadimos un guión delante del nombre del campo, es posible ordenar de forma invertida. Al hacer `-created`, le indicamos que nos muestre primero los proyectos de más actuales a más antiguos.

En cuanto a los nombres de los campos, también podemos utilizar el atributo `verbose_name` para cambiarlos:

```
class Project(models.Model):  
    title = models.CharField(max_length=200, verbose_name="Título" )  
    description = models.TextField( verbose_name="Descripción")
```

```
.....  
class Meta:  
    verbose_name = "proyecto"  
    ordering = ["-created"]  
  
def __str__(self):  
    return self.title
```

Campos especiales

Los campos de fecha y hora automatizados no aparecen en el administrador, Django los esconde para que no se puedan modificar, pero podemos mostrarlos como campos de tipo "sólo lectura".

Para hacerlo tenemos que extender un poco la configuración base del administrador de la siguiente forma:

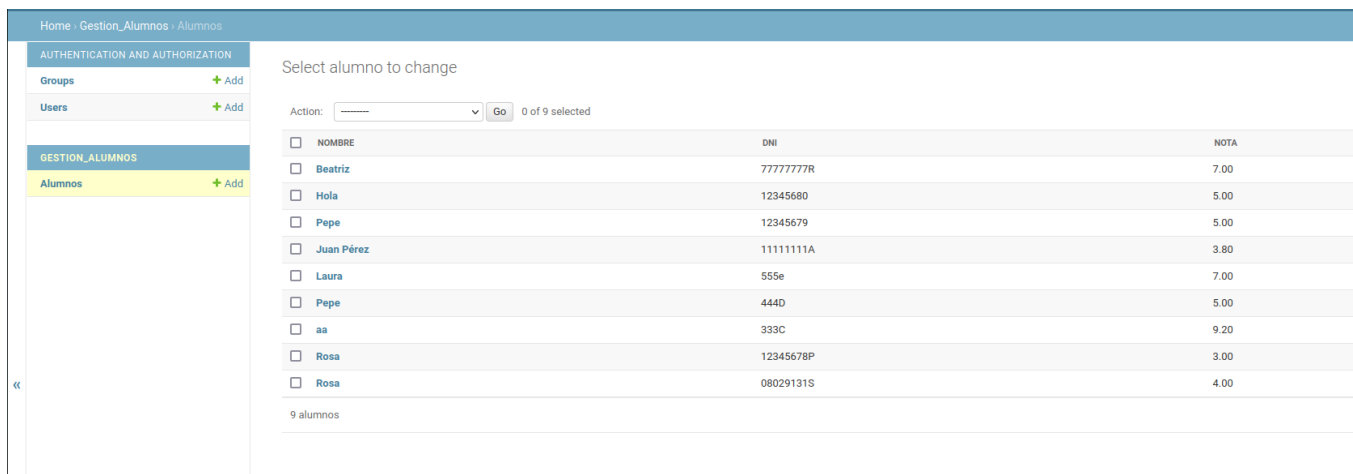
```
portfolio/admin.py  
  
from django.contrib import admin  
from .models import Project  
  
class ProjectAdmin(admin.ModelAdmin):  
    readonly_fields = ('created', 'updated')  
  
admin.site.register(Project, ProjectAdmin)
```

Lista de campos

Si yo quiero que aparezcan más campos tengo que utilizar la clase `ModelAdmin`. Hay que tener en cuenta que le preparamos el panel de administración a nuestro cliente porque será él quién haga uso de esta funcionalidad a través de esta opción y nosotros no se la programaremos.

```
class AlumnoAdmin(admin.ModelAdmin):  
    list_display= ("nombre", "dni", "nota")  
  
admin.site.register(Alumno, AlumnoAdmin)
```

Si refrescamos nuestro panel



NOMBRE	DNI	NOTA
Beatriz	77777777R	7.00
Hola	12345680	5.00
Pepe	12345679	5.00
Juan Pérez	11111111A	3.80
Laura	555e	7.00
Pepe	444D	5.00
aa	333C	9.20
Rosa	12345678P	3.00
Rosa	08029131S	4.00

9 alumnos

Otra opción bastante frecuente es la de buscar, podemos añadir una casilla de búsqueda a nuestro panel. Supongamos que tenemos en la tabla todos los alumnos del colegio, sería útil poderlos filtrar por etapa educativa, curso....

```
class AlumnoAdmin(admin.ModelAdmin):  
    list_display= ("nombre", "dni", "nota")  
    search_fields= ("nombre", "dni")  
  
admin.site.register(Alumno, AlumnoAdmin)
```

Esto quiere decir que podemos buscar por nombre o podemos buscar por dni

Select alumno to change

Q Rosa Search 2 results (9 total)

Action: 0 of 2 selected

☐	NOMBRE	DNI	NOTA
☐	Rosa	12345678P	3.00
☐	Rosa	08029131S	4.00

2 alumnos

Además de buscar podemos filtrar por un determinado campo, la diferencia es que si añadimos un filtro nos mostrará el panel los valores que tenemos almacenados para ese campo en nuestra tabla. Por ejemplo, si añadimos el ciclo a nuestros alumnos podemos filtrar por ese campo.

Vamos a hacerlo con la nota

```
class AlumnoAdmin(admin.ModelAdmin):
    list_display=("nombre","dni","nota")
    search_fields=("nombre","dni")
    list_filter=("nota",)

admin.site.register(Alumno,AlumnoAdmin)
```

En nuestro panel

Home > Gestion_Alumnos > Alumnos

AUTHENTICATION AND AUTHORIZATION

Groups + Add

Users + Add

GESTION_ALUMNOS

Alumnos + Add

Select alumno to change

Action: 0 of 3 selected

☐	NOMBRE	DNI	NOTA
☐	Hola	12345680	5.00
☐	Pepe	12345679	5.00
☐	Pepe	444D	5.00

3 alumnos

ADD ALUMNO +

FILTER

✖ Clear all filters

By nota

All

3.00

3.80

4.00

5.00

7.00

9.20

Es muy útil con las fechas porque al filtra en la parte FILTER del panel nos muestra opciones como Hoy, en los últimos 7 días, este mes, este año...

```
list_filter("fecha",)
```

Ponemos , al final porque espera una tupla. Se pueden poner varios filtros

Incluso puedo añadir la instrucción

```
date_hierarchy="fecha"
```

Y nos mostrará arriba los años que aparecen en nuestras fechas, al seleccionar año nos mostrará los meses....

Este tipo de opciones a nuestros clientes les vienen muy bien.

Site administration

AUTHENTICATION AND AUTHORIZATION		
Groups	+ Add	 Change
Users	+ Add	 Change

Por último, veamos como añadir usuarios con diferentes perfiles y por tanto los grupos de usuarios. Además de tener usuarios súperusuario(todos los permisos), podemos tener grupos de usuarios con distintos permisos según las operaciones que queramos dejarles hacer.

Por otro lado los usuarios podrán estar activos o no. Si los marcamos como no activos no podrán acceder a ninguna página de nuestra aplicación que requiera un login, no sólo al panel de administración

También podremos decidir si son staff o no. De no serlo no podrán acceder al panel de administración.

Para otorgar los mismos permisos a un conjunto de usuarios creamos grupos, elegimos los permisos por cada modelo y creamos usuarios dentro de esos grupos.

Para cada modelo hay tres permisos: añadir, editar, borrar